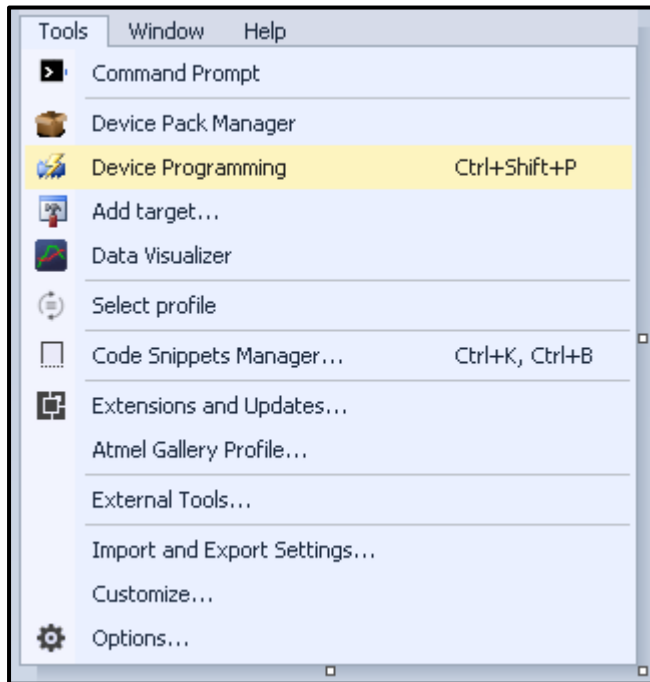


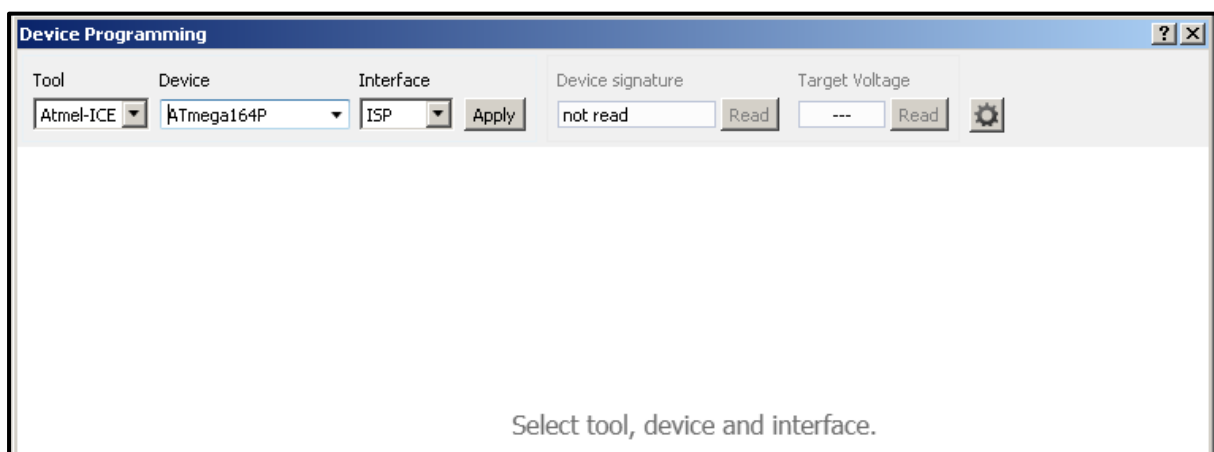
Setting AVR fuses in Atmel Studio 7

You will need an Atmel-ICE programmer connected to your PCB. If Atmel Studio prompts you to update the programmer's firmware (e.g. from 1.00 to 1.26), unplug the programmer from your PCB and proceed with the upgrade, or ask the technician before doing anything.

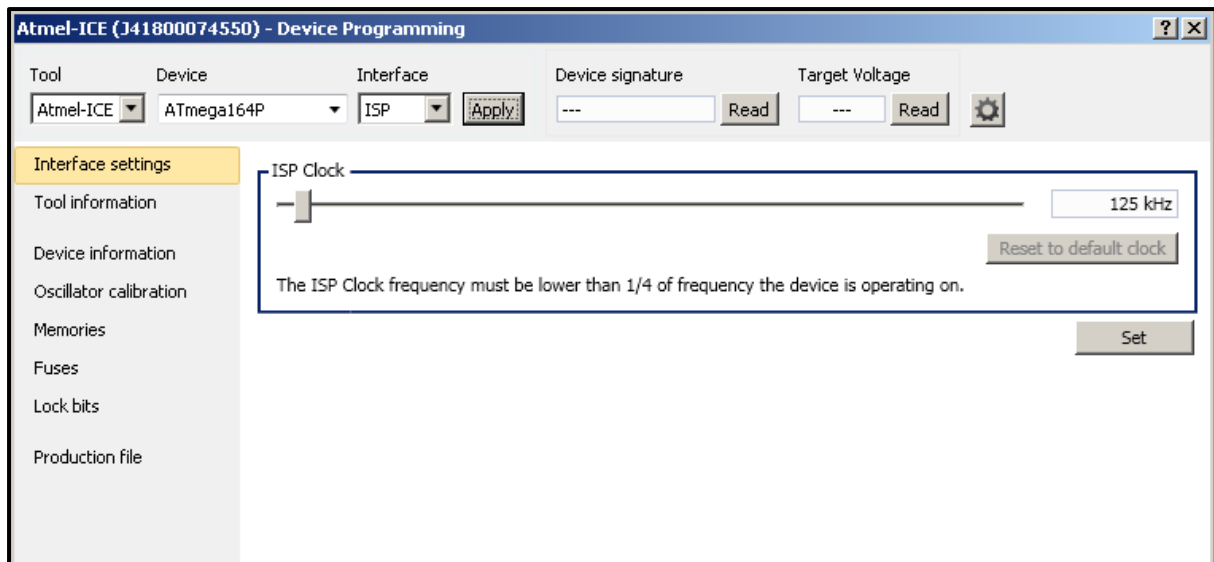
From the menu select **Tools** → **Device Programming** (shown below)



This will take you to this window:



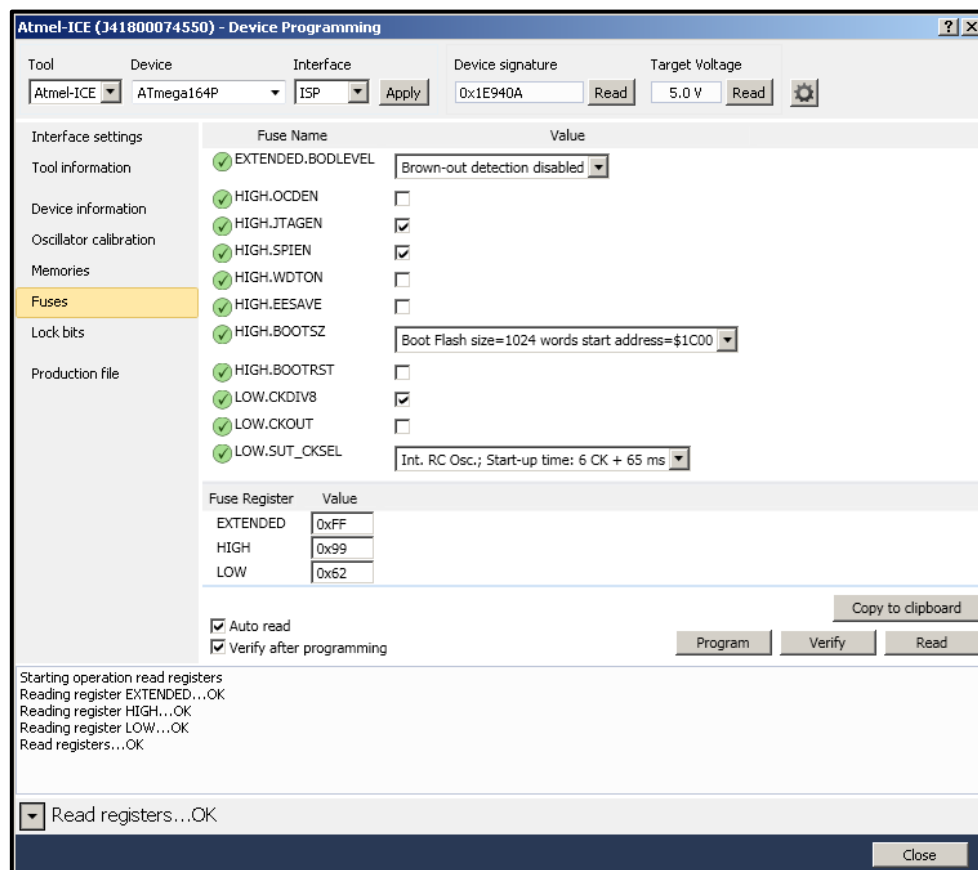
Atmel Studio should see the programmer (**tool**), device and programming interface. If not, set it to the above and click **Apply**. The window should change to look like this:



Do not change the ISP clock frequency.

To check communication, read the **Device signature**. An ATmega164P should return **0x1E940A**, and the **Target Voltage** should match your chosen VCC.

Select **Fuses** on the left-hand menu. An out-of-the-box '164P should look like this:



Top to bottom:

BODLEVEL – **B**rown-**O**ut **D**etection **LE**vel. A brown-out is a drop in supply voltage (as opposed to a complete power cut). The AVR can be set to trigger an interrupt if the supply drops below the selected value. Leave this disabled unless you actually need to make use of it.

OCDEN – **O**n-**C**hip **D**ebug **E**nable. The programming interface we use doesn't support this, so leave it unchecked

JTAGEN – **J**TAG **E**nable. An alternative programming interface. This takes up half of PORTC, so should be disabled

SPIEN – Serial Programming **E**nable via the **SPI** port. This is what the programmers we have use to program the AVR. If you disable this, the AVR will be 'bricked' and need to be relocated to the bin.

WDTON – **W**atch**D**og **T**imer always **ON**. Similar to the brown-out detection, only enable the watchdog timer if you're using it.

EESAVE – Preserves EEPROM data when erasing and programming. Only enable this if you are using the EEPROM and need it to preserve its data.

BOOTSZ – **BO**OT flash **Si**Ze. Only applicable if you're writing a bootloader. If you are, you'll know how much flash you want reserved

BOOTRST – **BO**OT **Re**Se**T** vector enabled. Used in conjunction with BOOTSZ

The final three fuses deal with the system clock, which needs a description.

When the AVR's are shipped from the factory, they are running from an internal RC-based oscillator, nominally (i.e. it's not very accurate) running at 8MHz. The very first AVR, the AT90S1200 had an internal 1MHz clock, so all AVR's run at 1MHz by default. This is achieved by having an internal divide-by-eight circuit after the main clock. The system clock can also be fed to an output (PORTB bit 1 in this case).

CKDIV8 – **C**lock **DIV**ide by **8**. With this fuse set, the system clock (whether it is internal or external, such as a crystal) is divided by eight before being fed to the processor. For example, if you had a 12MHz crystal as your system clock and CKDIV8 is set, your AVR will actually be running at 1.5 MHz (12MHz / 8).

You will rarely need this fuse set.

CKOUT – **C**lock **OUT**. This routes the AVR clock to PORTB bit 1 (pin 2 on the DIP version of the '164P). Using the same example as above, a 1.5MHz square wave would appear on pin 2.

You will rarely need this fuse set.

SUT_CKSEL – **Start-Up Time Clock SElect**. This has many options, and you need to choose the correct one. If you select an external crystal as the clock source and there is no crystal, you will not be able to reprogram it very easily.

Each entry has three things. The first is the clock source, and can be

- The internal 8MHz RC oscillator,
- The internal 128kHz oscillator,
- An external low-frequency oscillator (typically a 32.768kHz watch crystal),
- External ceramic resonators (cheaper than crystals but not as accurate),
- External crystals of assorted frequency ranges. The lower frequency crystal setting use a low-power circuit often called a Vittoz oscillator.

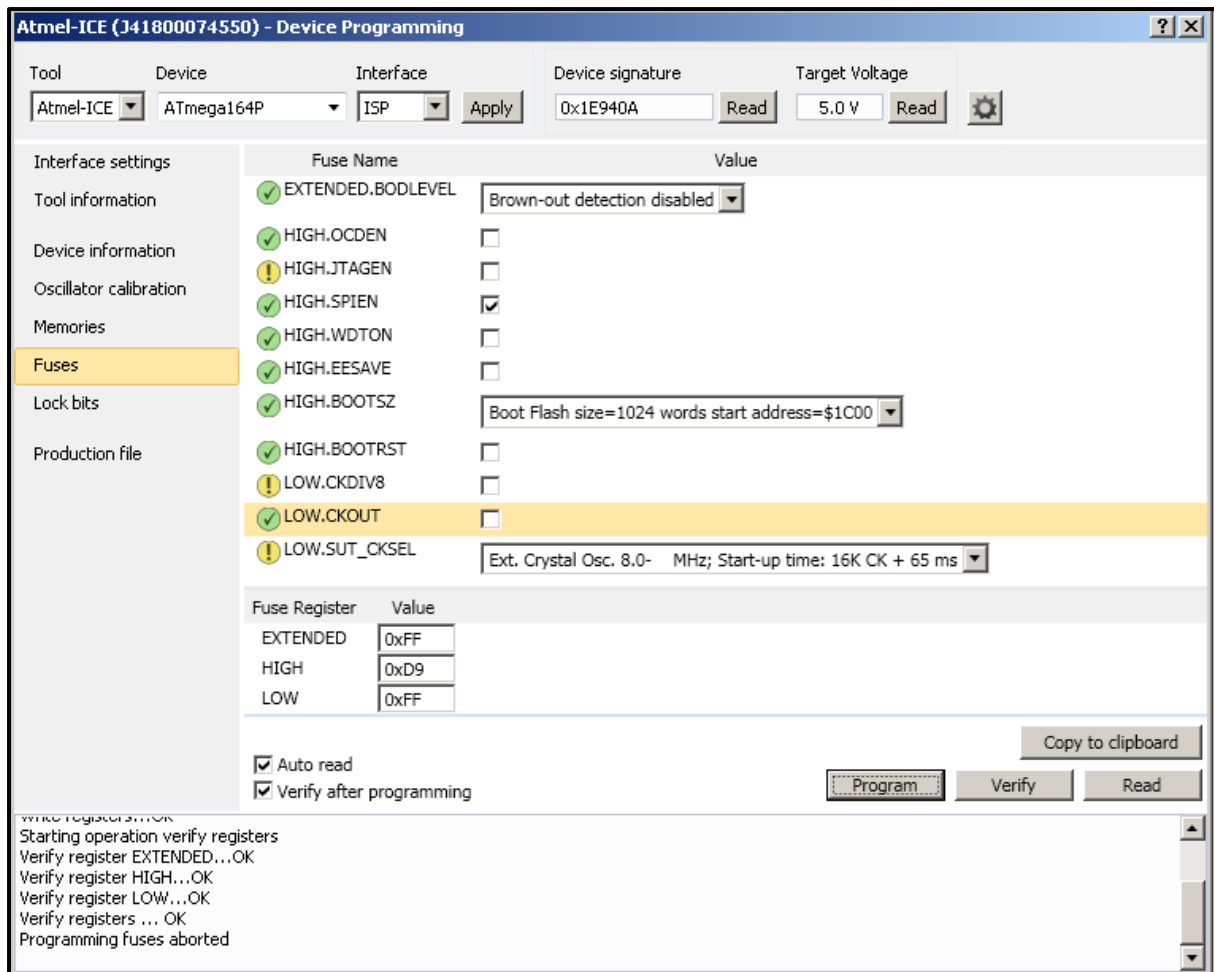
If you are doing any form of timing-critical work, such as serial communication through the UARTs, you will be using one of the options that starts **Ext. Crystal Osc. 8.0-**. This is for external crystals from 8MHz to 20MHz.

The next item is the start-up time, which has two parts. These both form a delay between power being applied and the CPU starting to allow the oscillator to stabilise before any code is executed. For example, 1K CK + 65 ms will delay the processor starting by 1000 clock cycles + 65 ms.

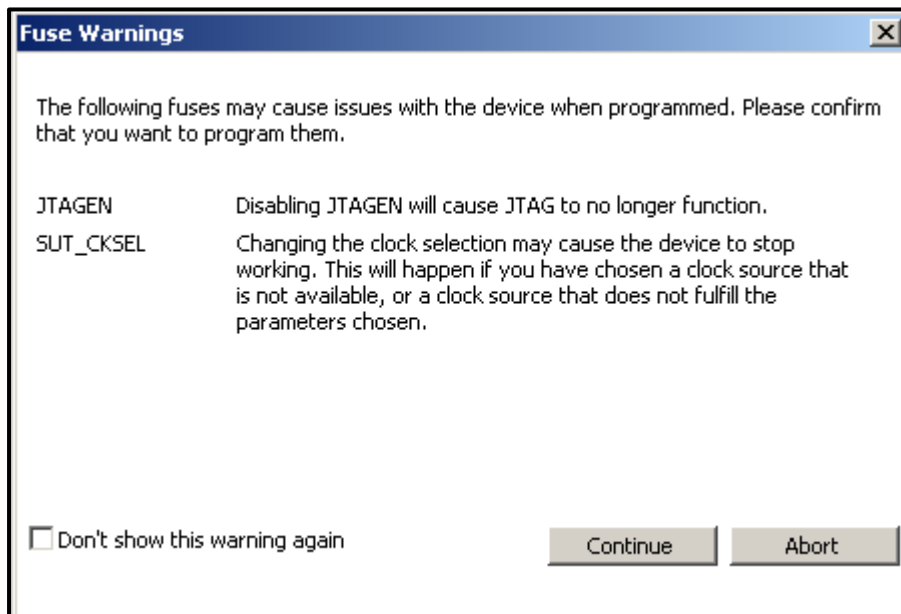
The simplest way to approach it is to pick the longest start-up time you can afford. For projects, that's the 16K CK + 65 ms option.

When you change any fuse settings, the green tick will be replaced with a yellow exclamation mark. This is to tell you that the option has been changed but not programmed.

For the project PCB, select the settings shown below:



When you click **Program**, you should see a warning like this:



This is expected, so click **Continue**. The fuses will be programmed. If it fails, ask a demonstrator for help.

To be sure, read the device's signature to check communication is still possible (i.e. check this AVR isn't bricked).

At this point, your AVR should be running at the correct frequency and you'll not need to change the fuses again.